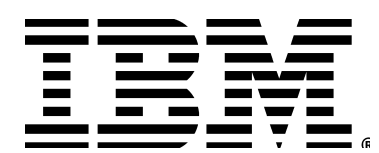
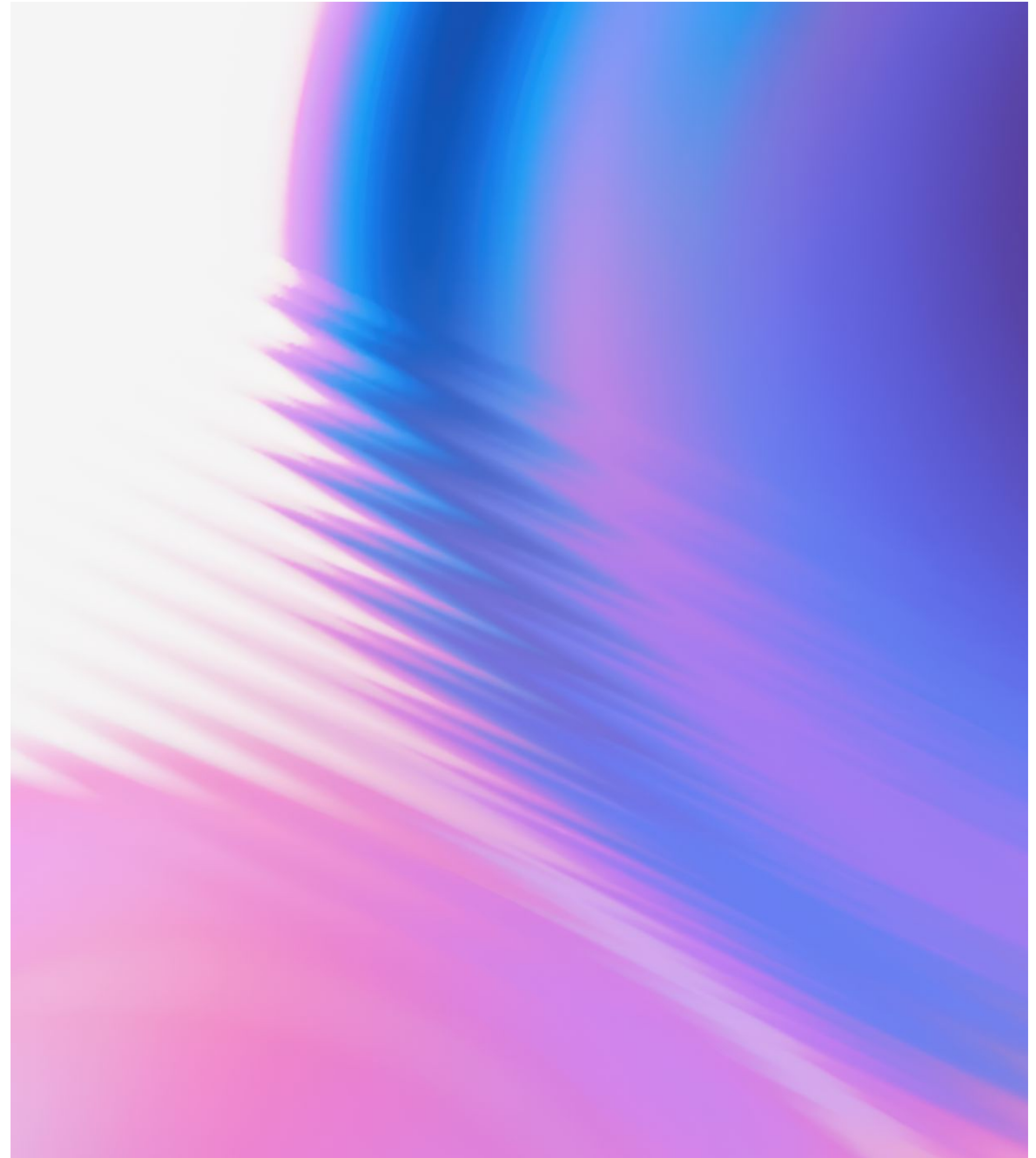


Quantum puzzles on utility scale devices

James R. Wootton
IBM Quantum



c



Resurrecting work from 6 years ago

arXiv > quant-ph > arXiv:1806.02736

Quantum Physics

[Submitted on 7 Jun 2018]

Benchmarking of quantum processors with random circuits

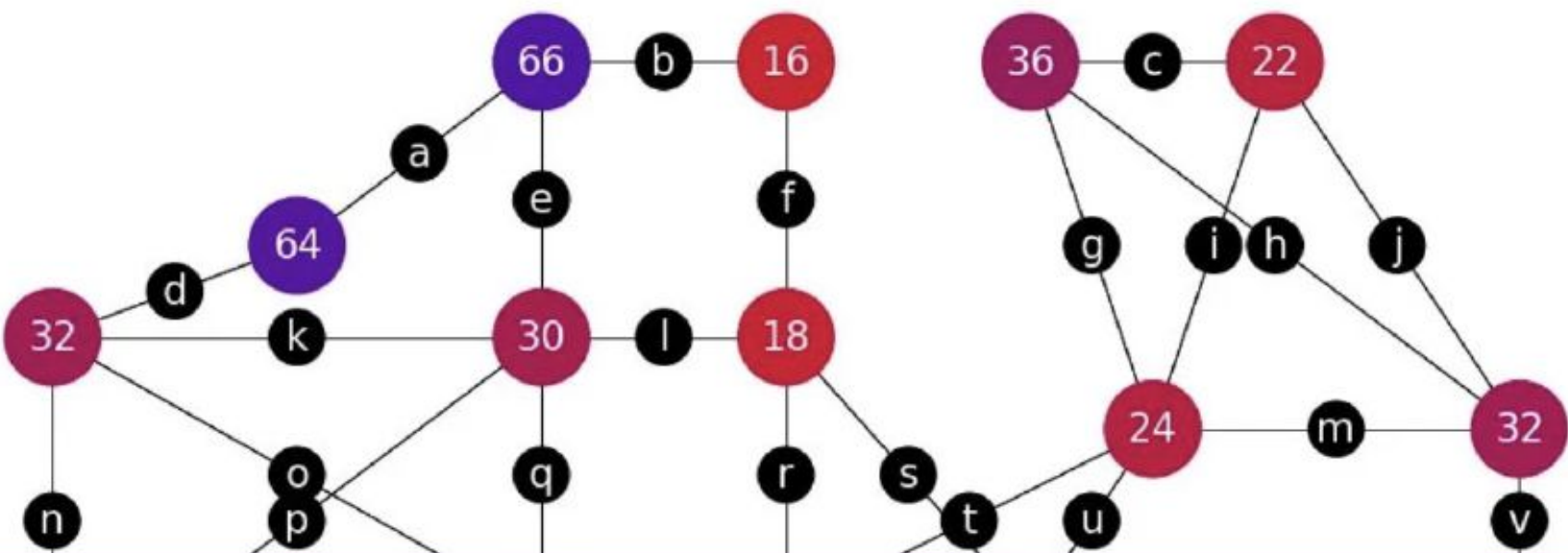
James R. Wootton

Quantum processors with sizes in the 10–100 qubit range are now increasingly common. However, with increasing complexity for benchmarking. The effectiveness of a given device may vary greatly between different tasks, from single and two qubit gate fidelities. For this reason, it is important to assess processor quality for a range of tasks. We propose and implement tests based on random quantum circuits. These are used to evaluate multiple different devices with sizes from 5 to 19 qubits, from two hardware manufacturers: IBM Research and Rigetti. The data is analysed and how the devices perform. We also describe how it can be used for a qualitative description accessible to the general public.

Using a simple puzzle game to benchmark quantum computers

Dr James Wootton
7 min read · Jan 16, 2018

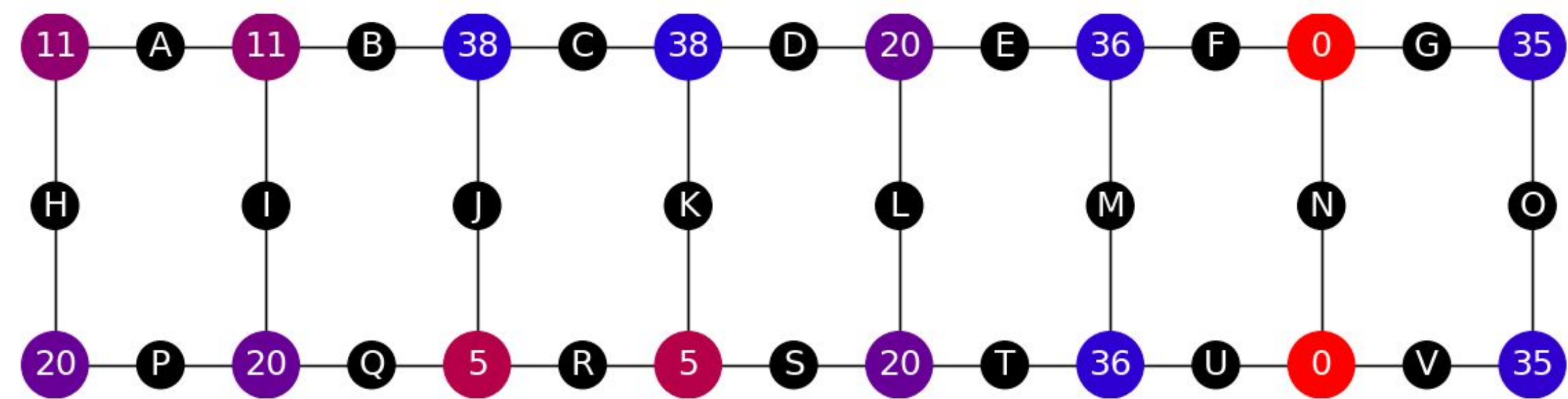
209 1



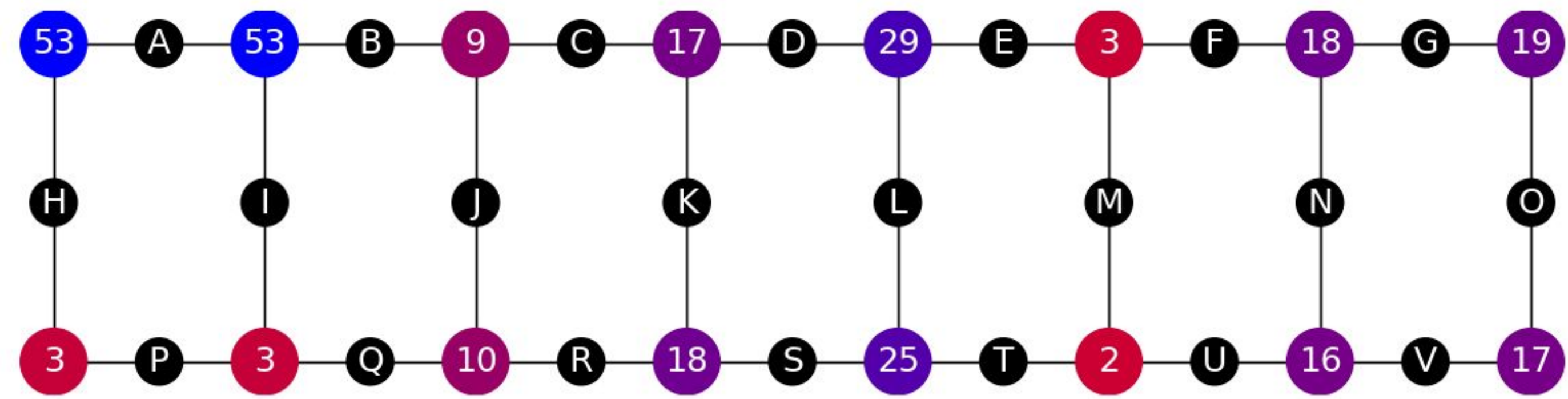
Puzzles on a coupling graph

- We will play a game on a quantum device
 - A random disjoint set of pairs is selected from the coupling graph
 - Each pair is given a random number, displayed on each qubit
- Aim of the game: look at the numbers and figure out the pairing
- In later rounds, the numbers get perturbed to increase the difficulty

Round 1



Round 4



Circuit for round 1

- Numbers are generated by a quantum circuit
 - For each pair we choose a random angle and entangle them as follows

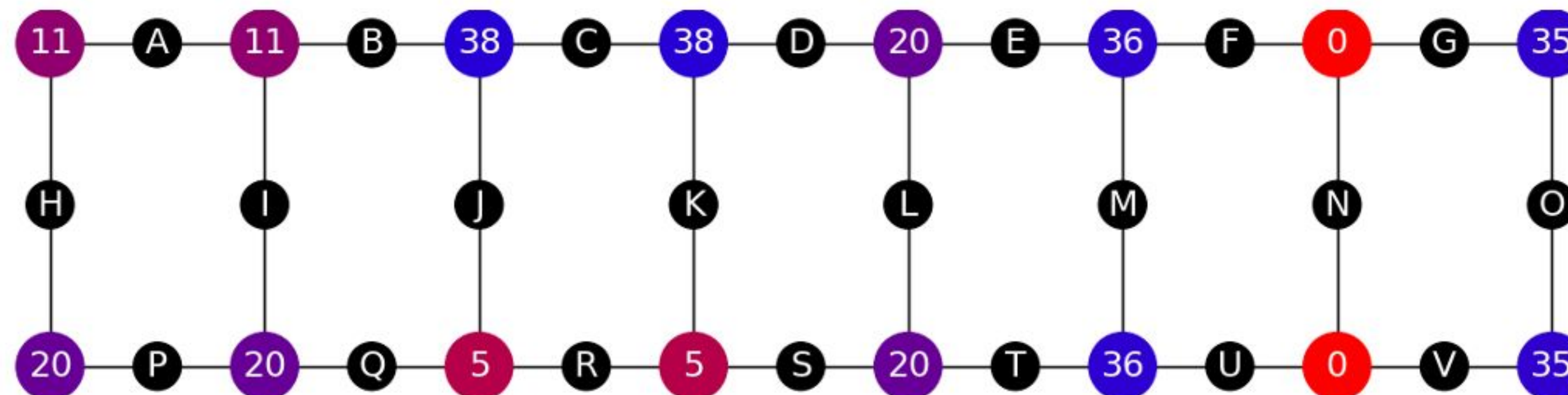
```
qc.cx(j,k)
qc.rx(theta,j)
qc.cx(j,k)
```

$$\cos\frac{\theta}{2}|00\rangle - i\sin\frac{\theta}{2}|11\rangle$$

- The probability of a **1** outcome on each qubit is then

$$p_j = p_k = \sin^2\frac{\theta}{2}$$

- This is the same for the two qubits of each pair, but varies from pair to pair
- The probability of a **1** is then the number reported



Circuit for round 2+

- In theory:

- First remake round 1:

```
qc.cx(j,k)
qc.rx(theta,j)
qc.cx(j,k)
```

- Then unmake it:

```
qc.cx(j,k)
qc.rx(-theta,j)
qc.cx(j,k)
```

- Then generate a new random pairing and entangle those pairs

- The remake/unmake rounds are also conjugated by random `rx` or `ry` rotations

- Buildup of errors causes the numbers to drift from their correct values

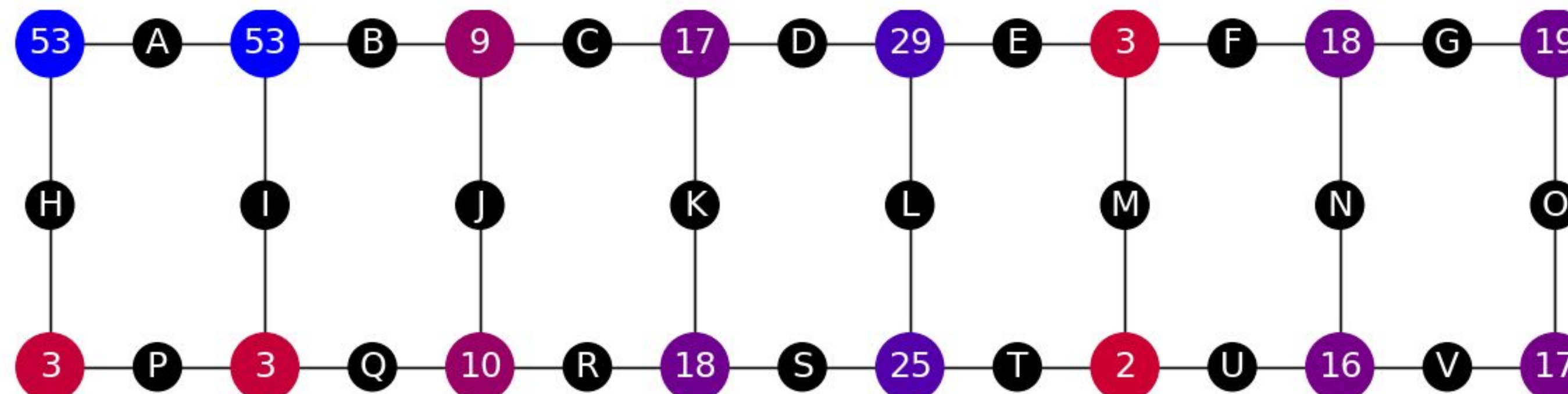
- In practice:

- For the pairs of round 1, apply

```
qc.cx(j,k)
qc.u(0,-pi/2,pi)
qc.cx(j,k)
```

(which is equivalent to the identity)

- Then generate a new random pairing and entangle those pairs



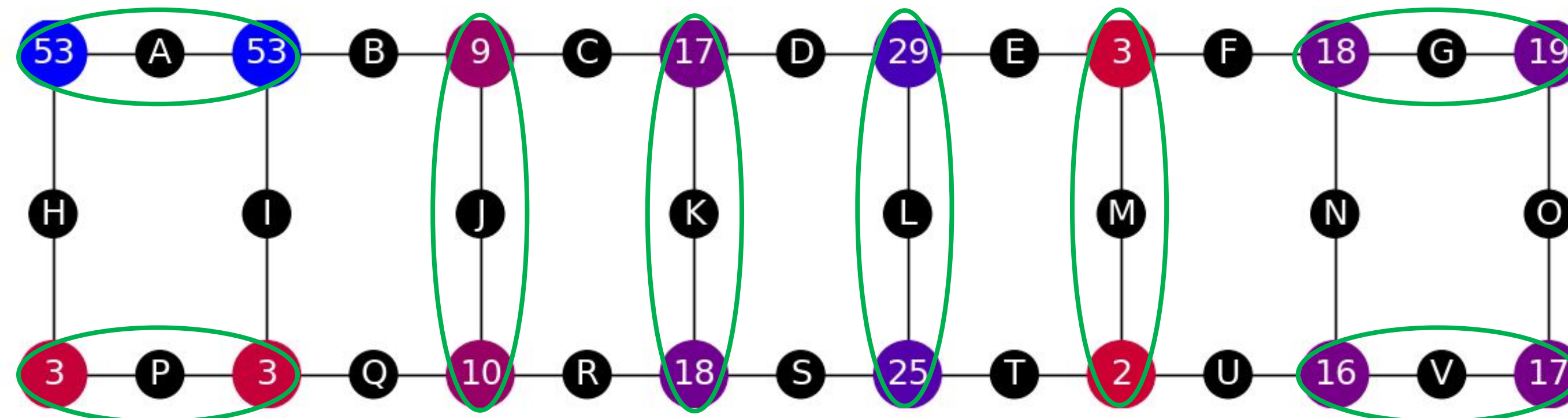
Quantifying awesomeness

- With many samples, how can we quantify quality?
- Fuzz:
 - For the two qubits in each pair, how different are their values?
- Difference
 - For each qubit, how different is its value from what it should be?
- Matching success:
 - Using an algorithm to guess the pairing, how often is it right?

$$|p_j - p_k|$$

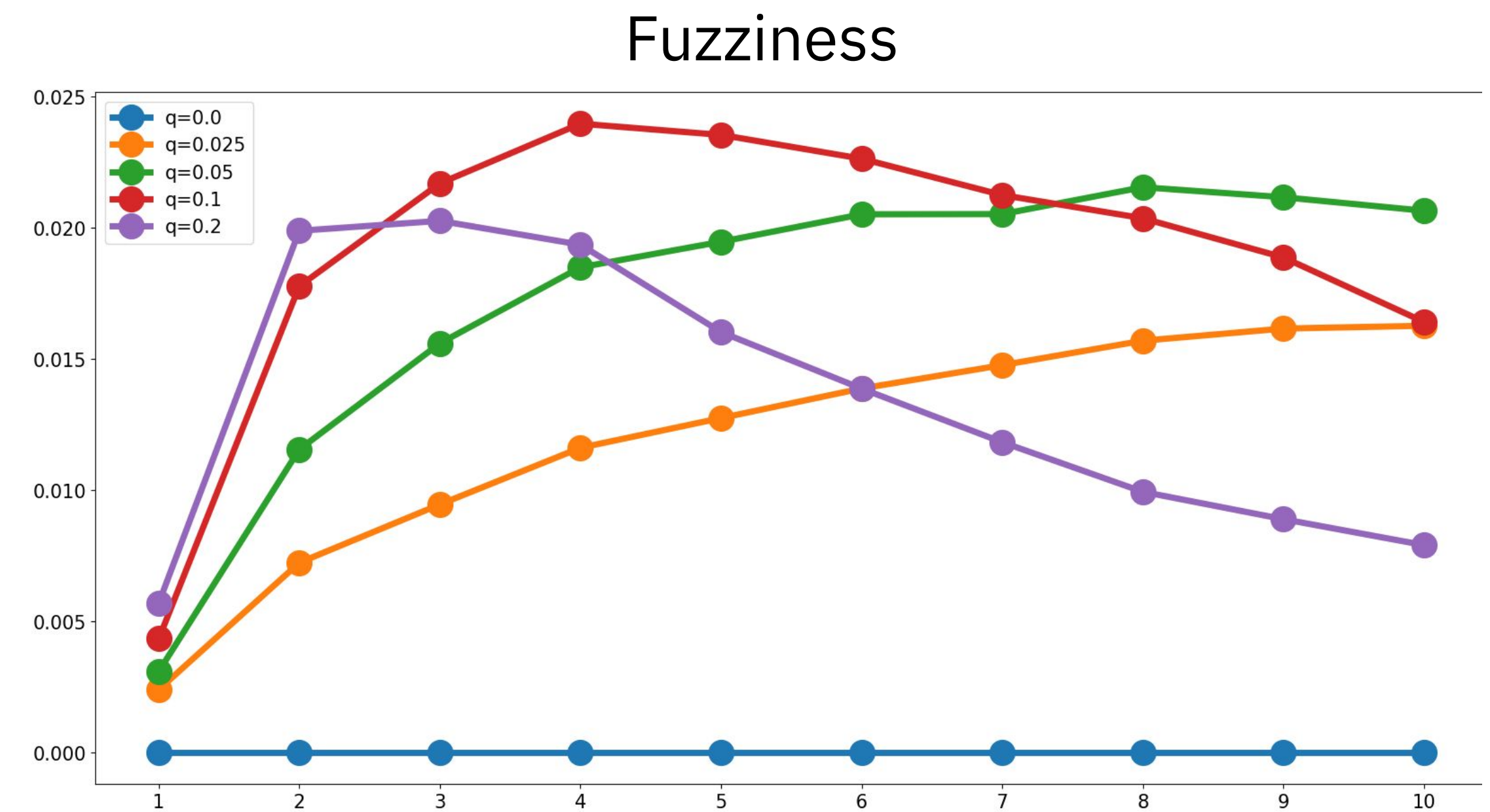
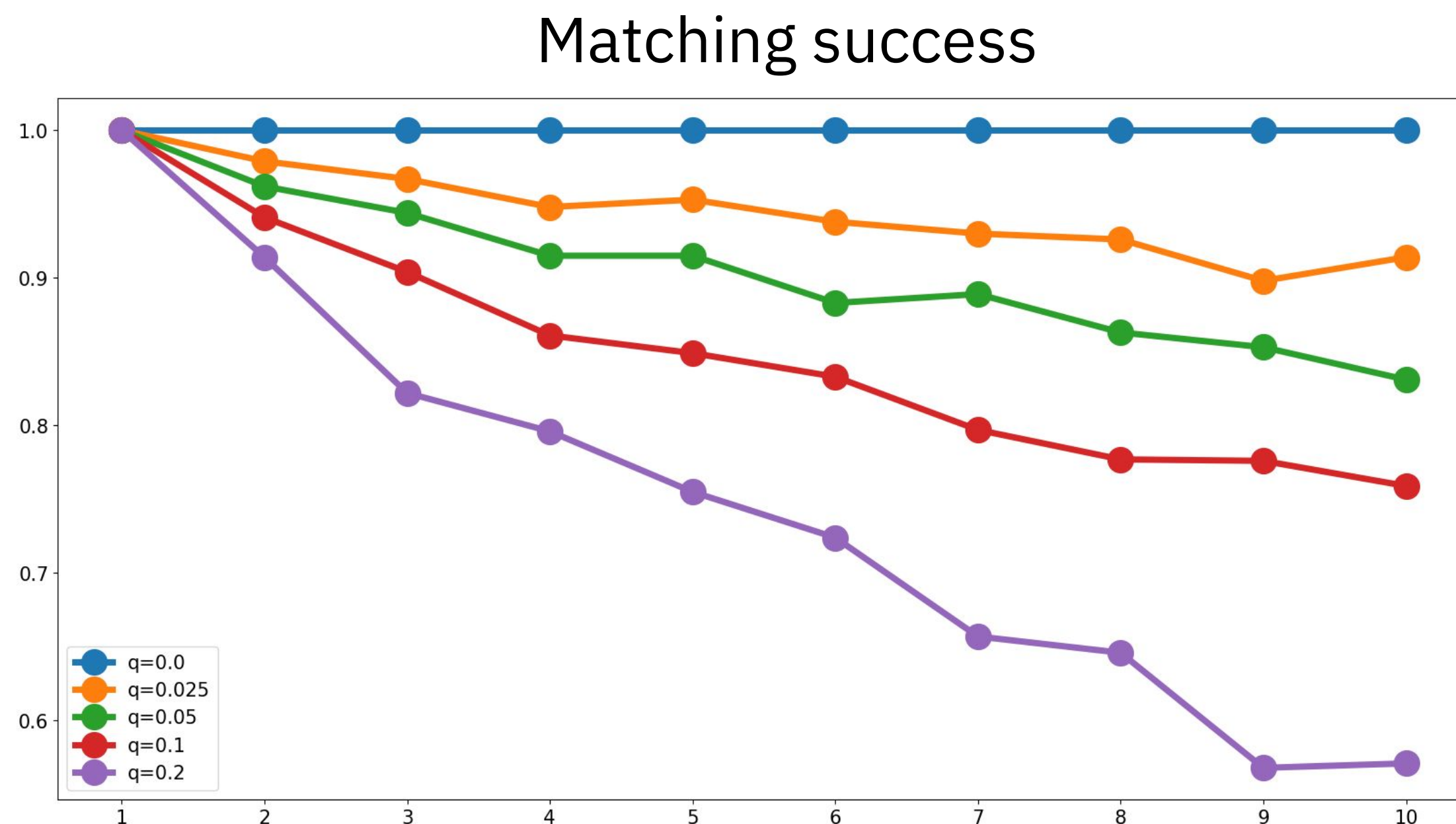
$$\left| \frac{2 \arcsin \sqrt{p_j}}{\pi} - \theta \right|$$

c

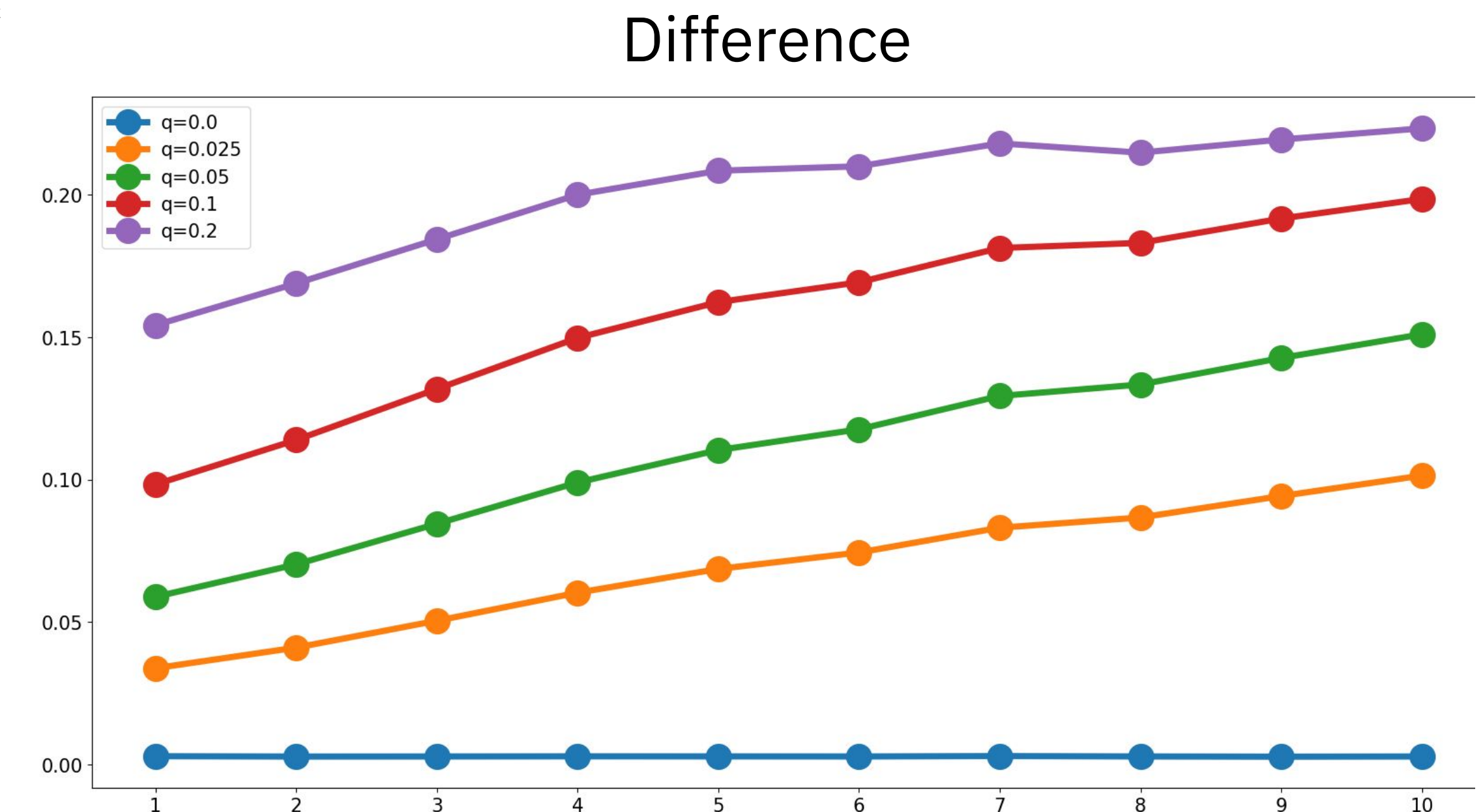


Results from noisy simulations

- Simulations of 5 qubit `ibmqx4`
- Depolarizing noise on every gate with probability p
- For the fuzziness we get the ‘peak of doom’
 - Low fuzziness at the beginning because all is well
 - Low fuzziness at the end because everything is awful
 - In the middle we get a landmark



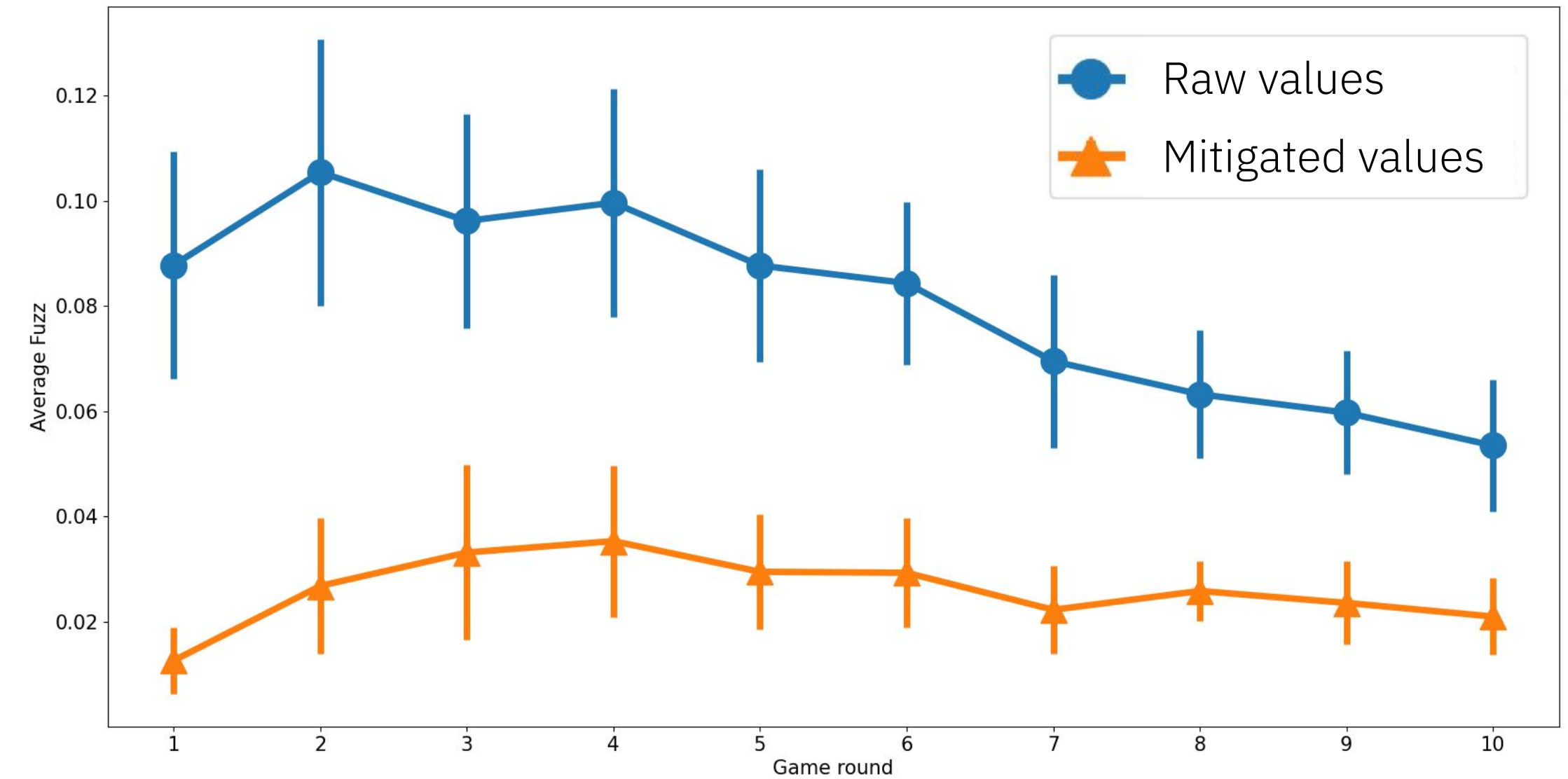
C



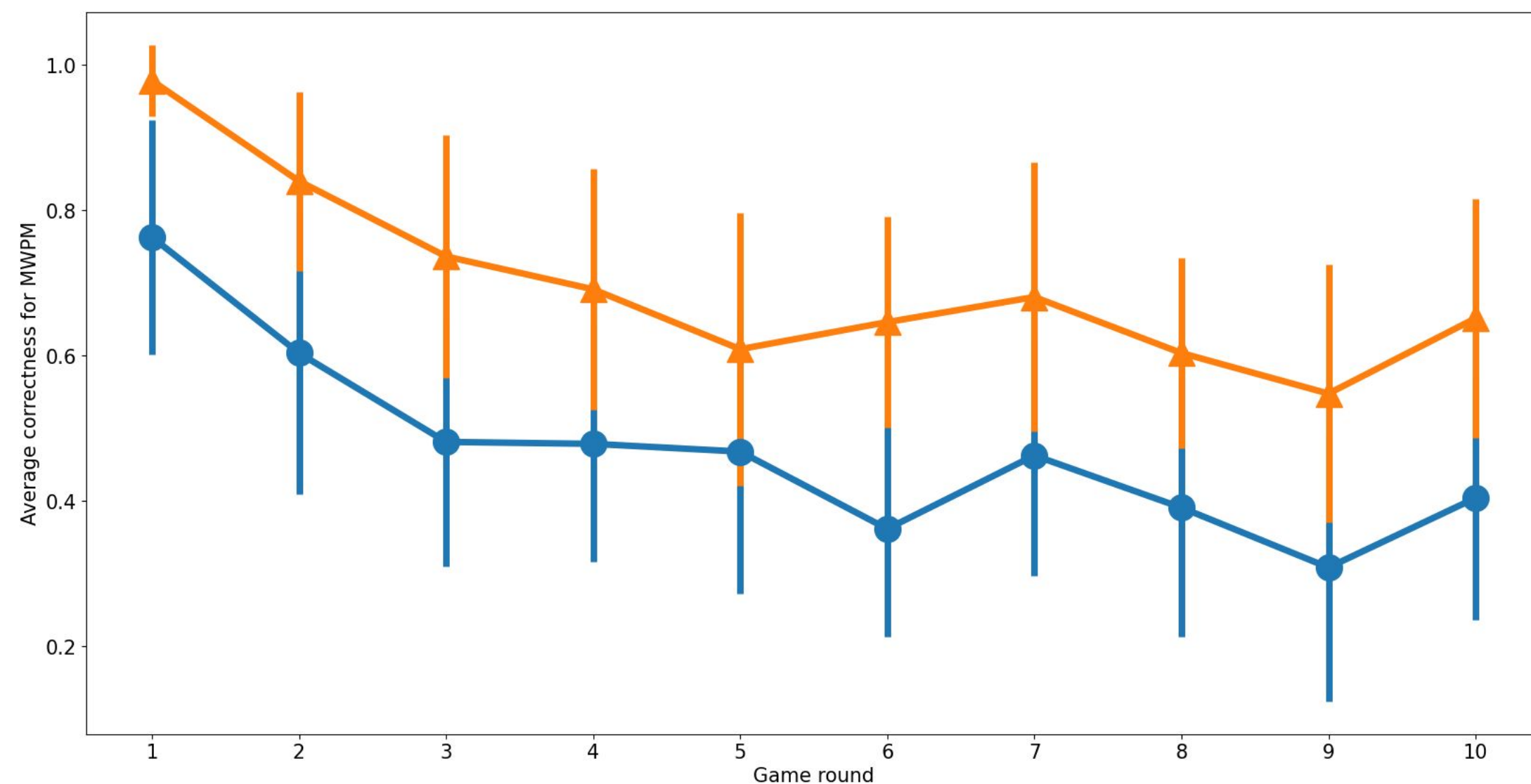
Results from `ibm_rueschlikon`

- 16 qubit device (also known as `ibmqx5`)
- Peak of doom already at ~round 2
 - CX depth of 4
- Matching success and difference converged by ~round 10
- Great for its time, but not really very good

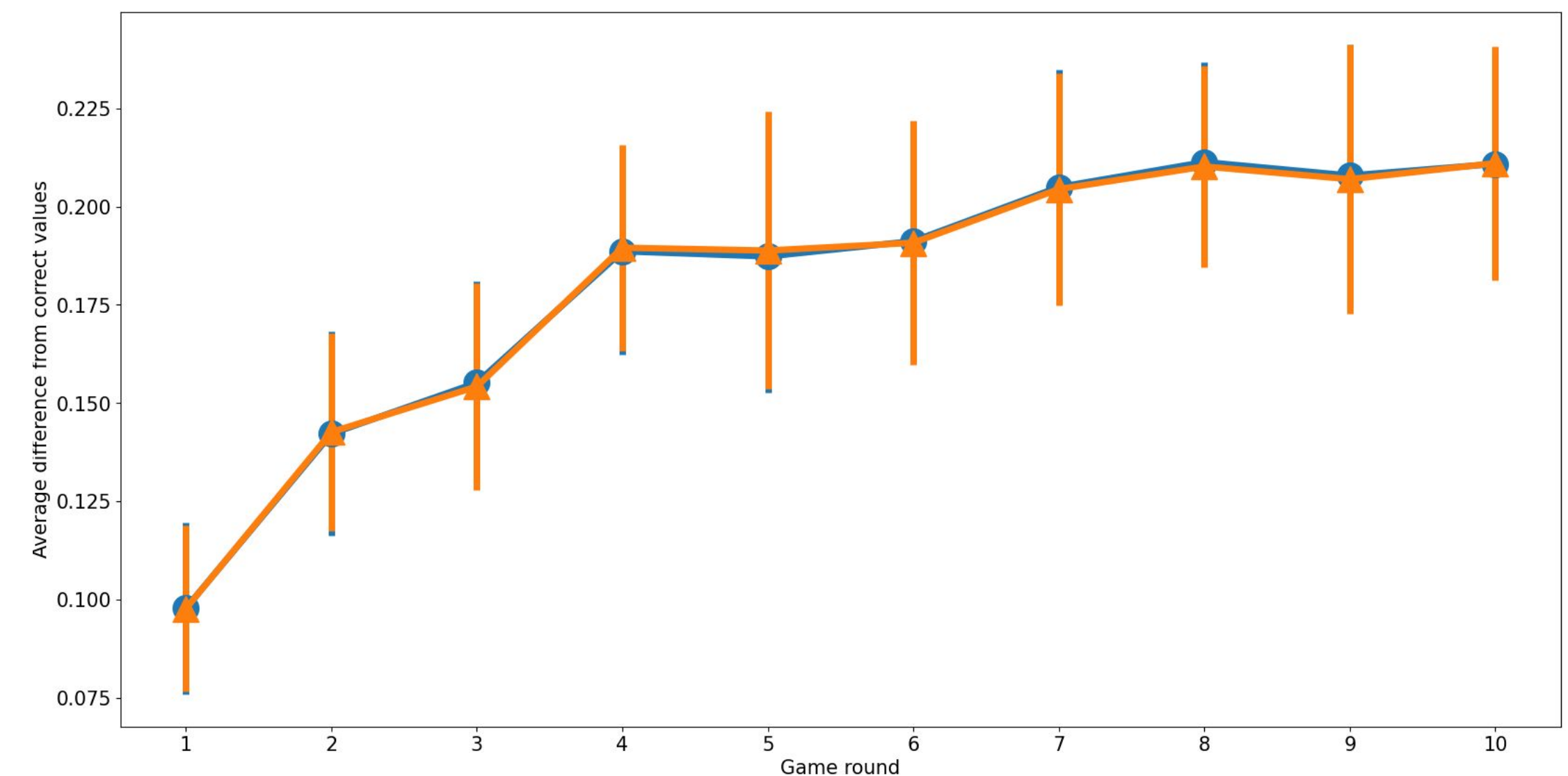
Fuzziness



Matching success



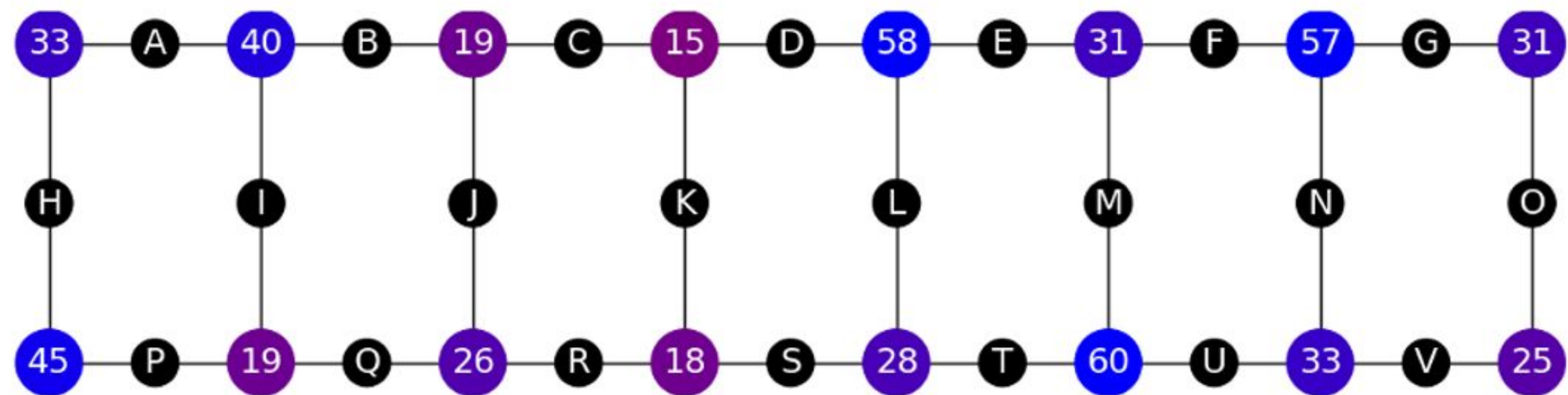
Difference



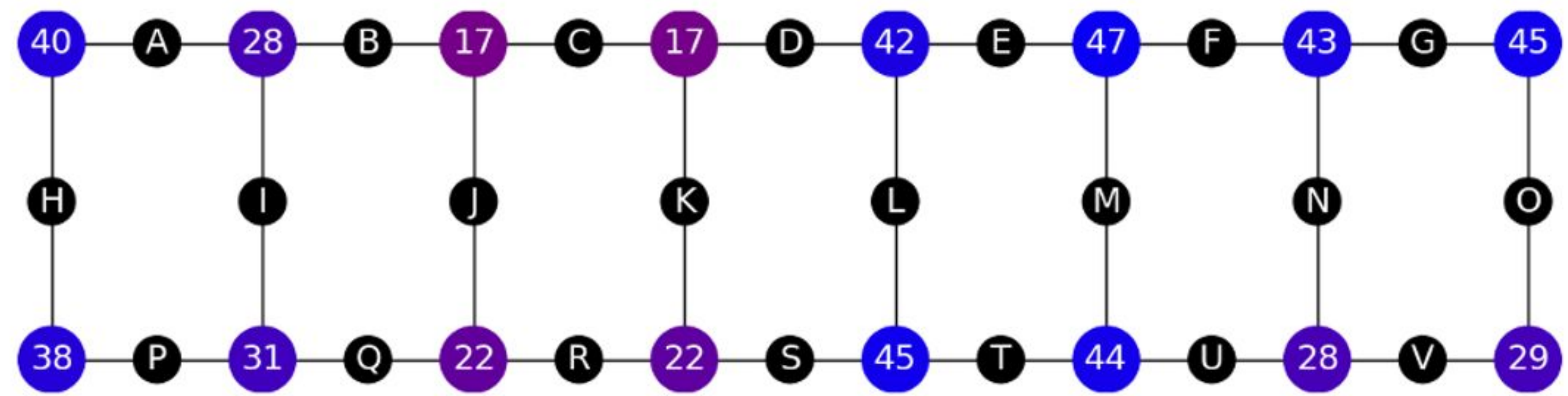
Puzzles from `ibm_rueschlikon`

- This is also seen from the puzzles. Here's an attempt at round 2
- Correct solution: C, H, I, R, L, M, N, O
- Mitigation helps a bit
- But it still doesn't give very good results

Round 2



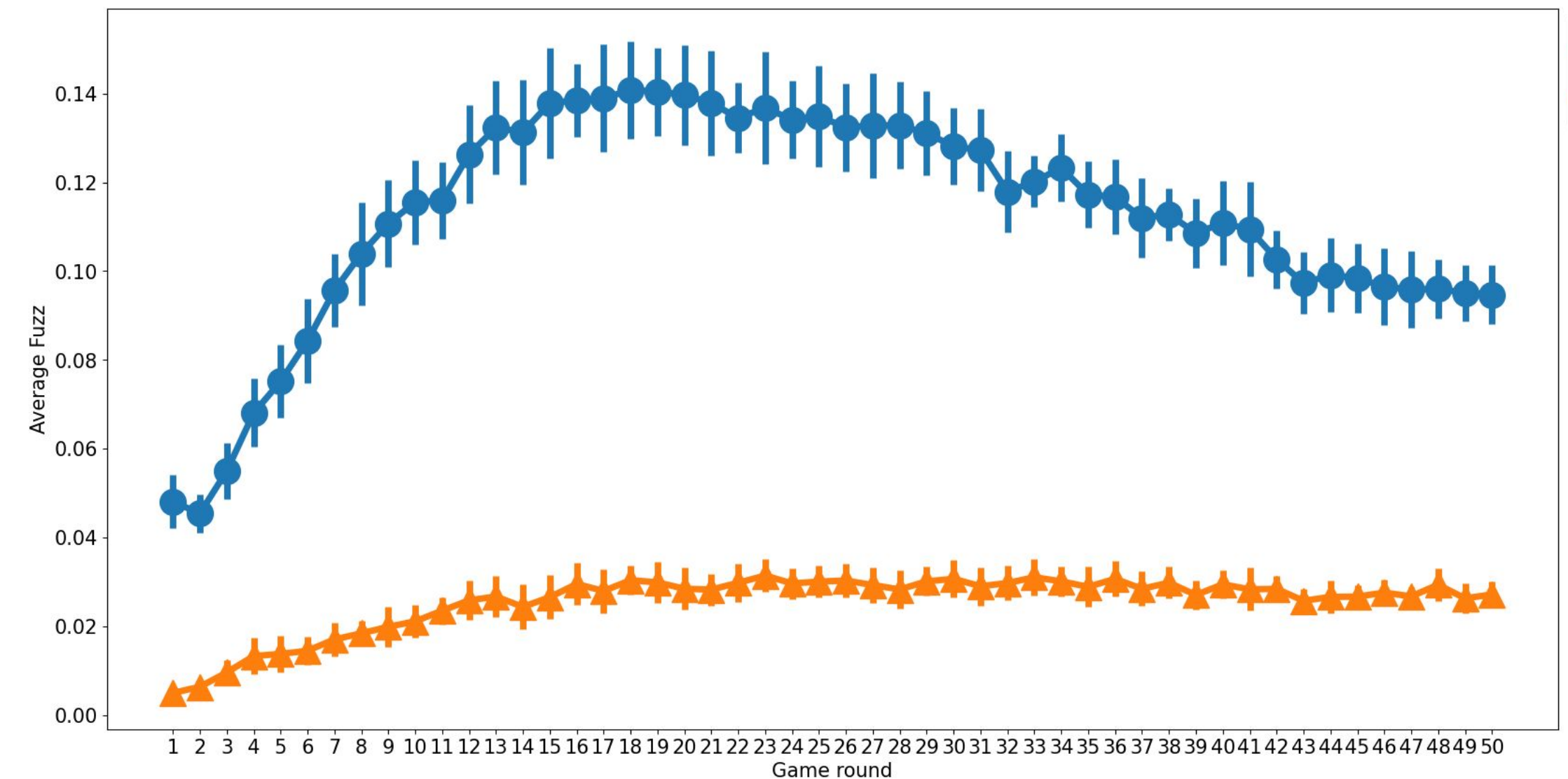
Round 2 (cleaned up)



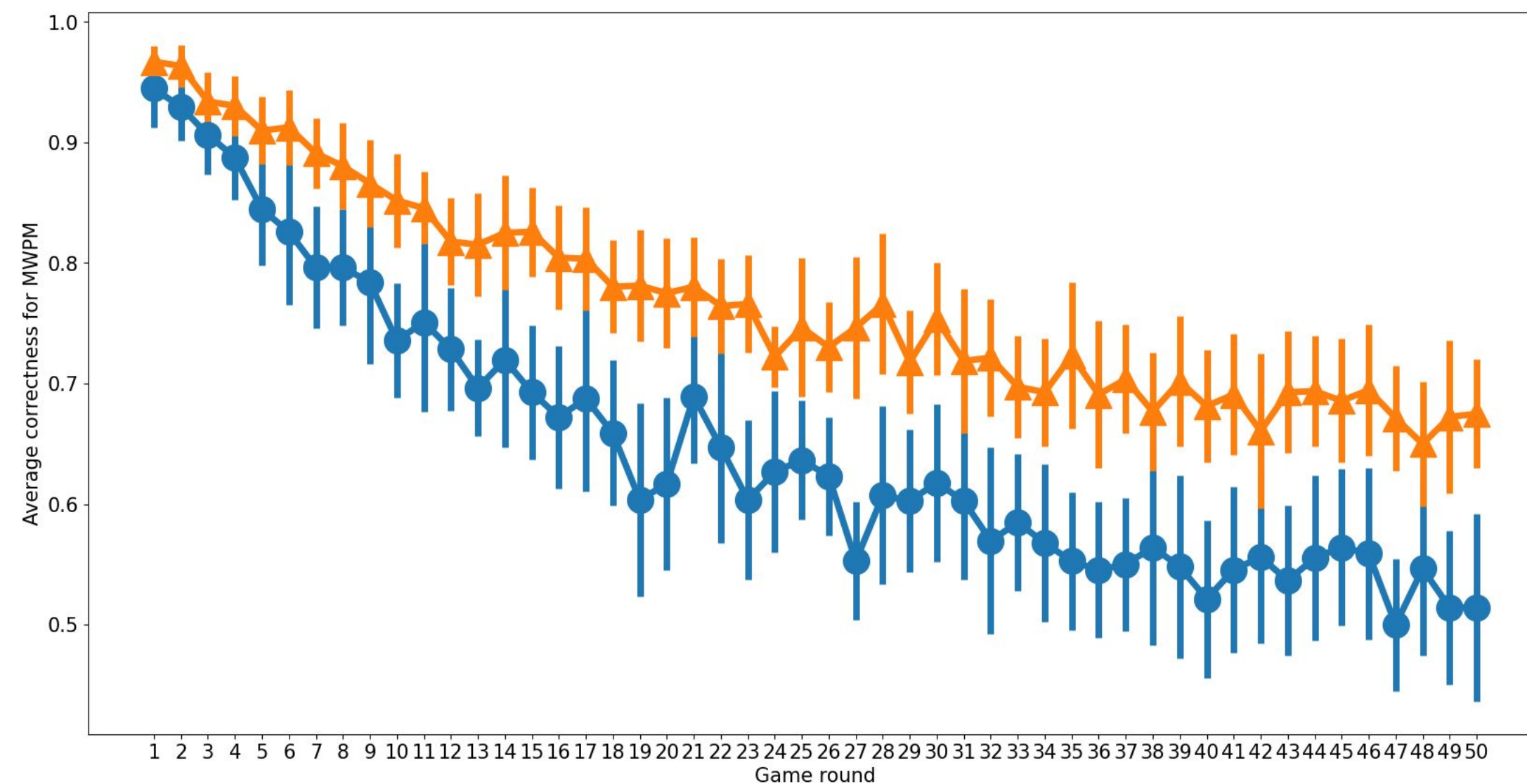
Results from `ibm_torino`

- 133 qubit *Heron* device
- Best ELPG at time of running
- Matching success and difference starting to converge
- Much better than `ibmq_rueschlikon`

Fuzziness

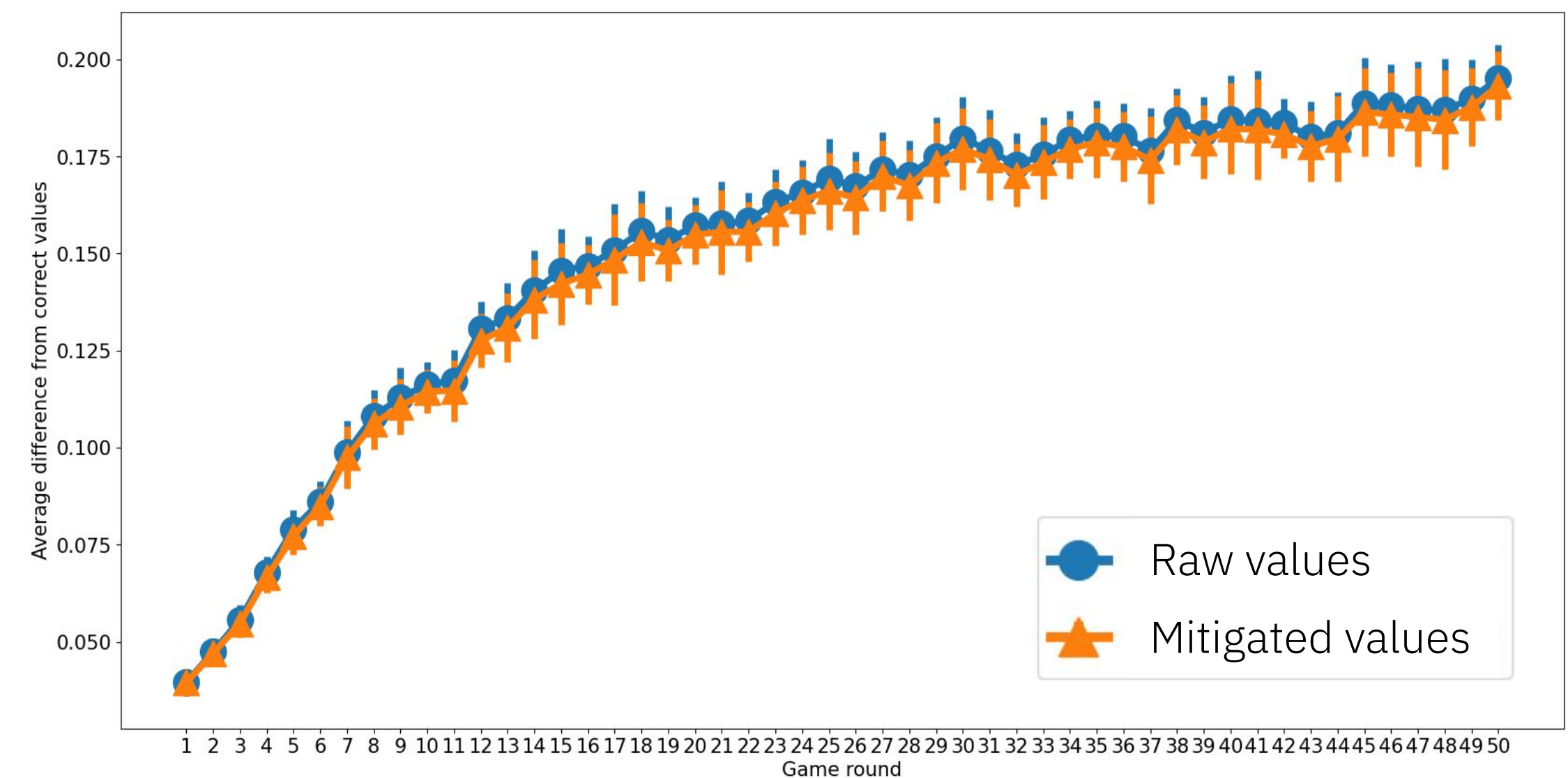


Matching success



c

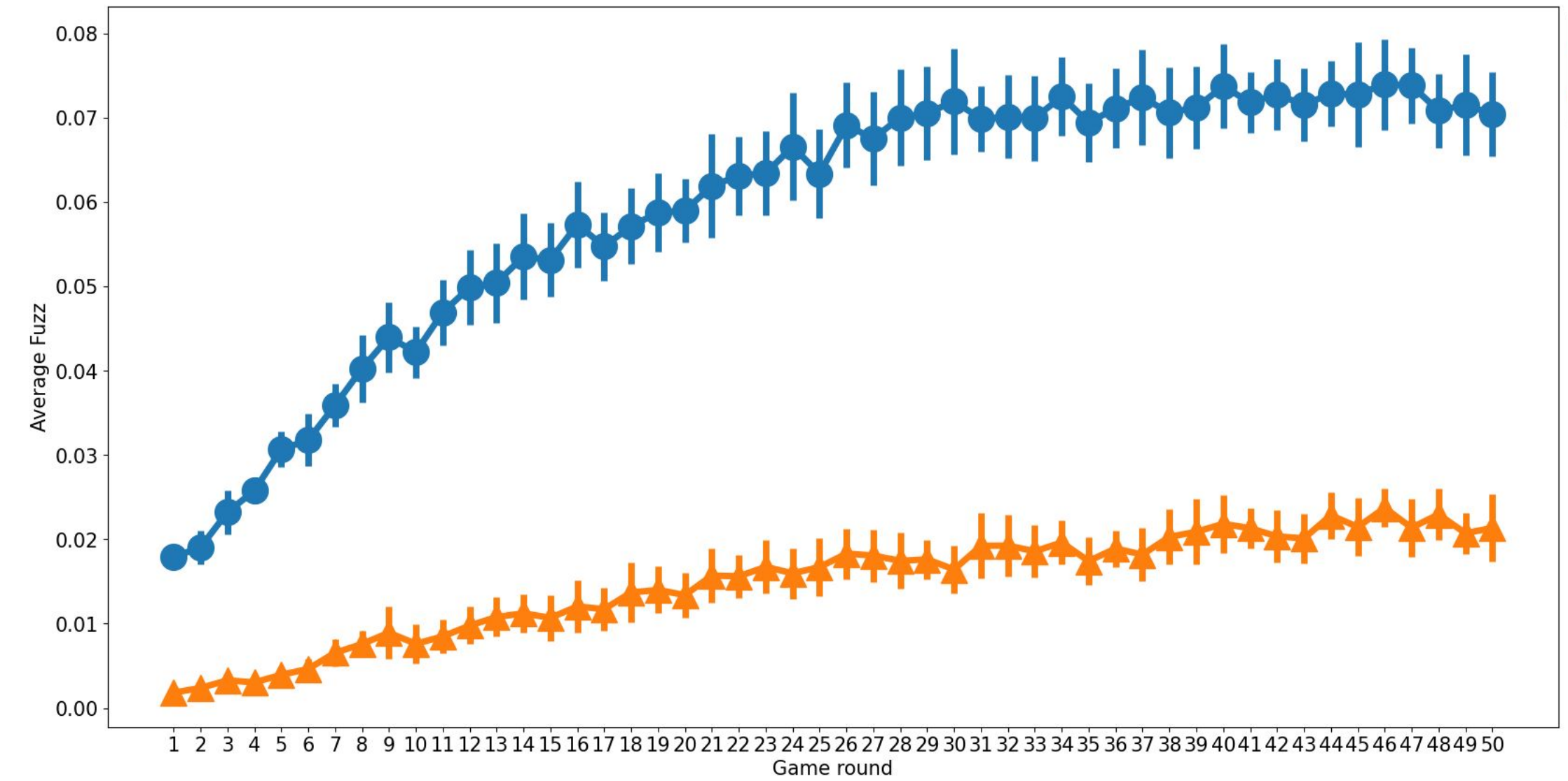
Difference



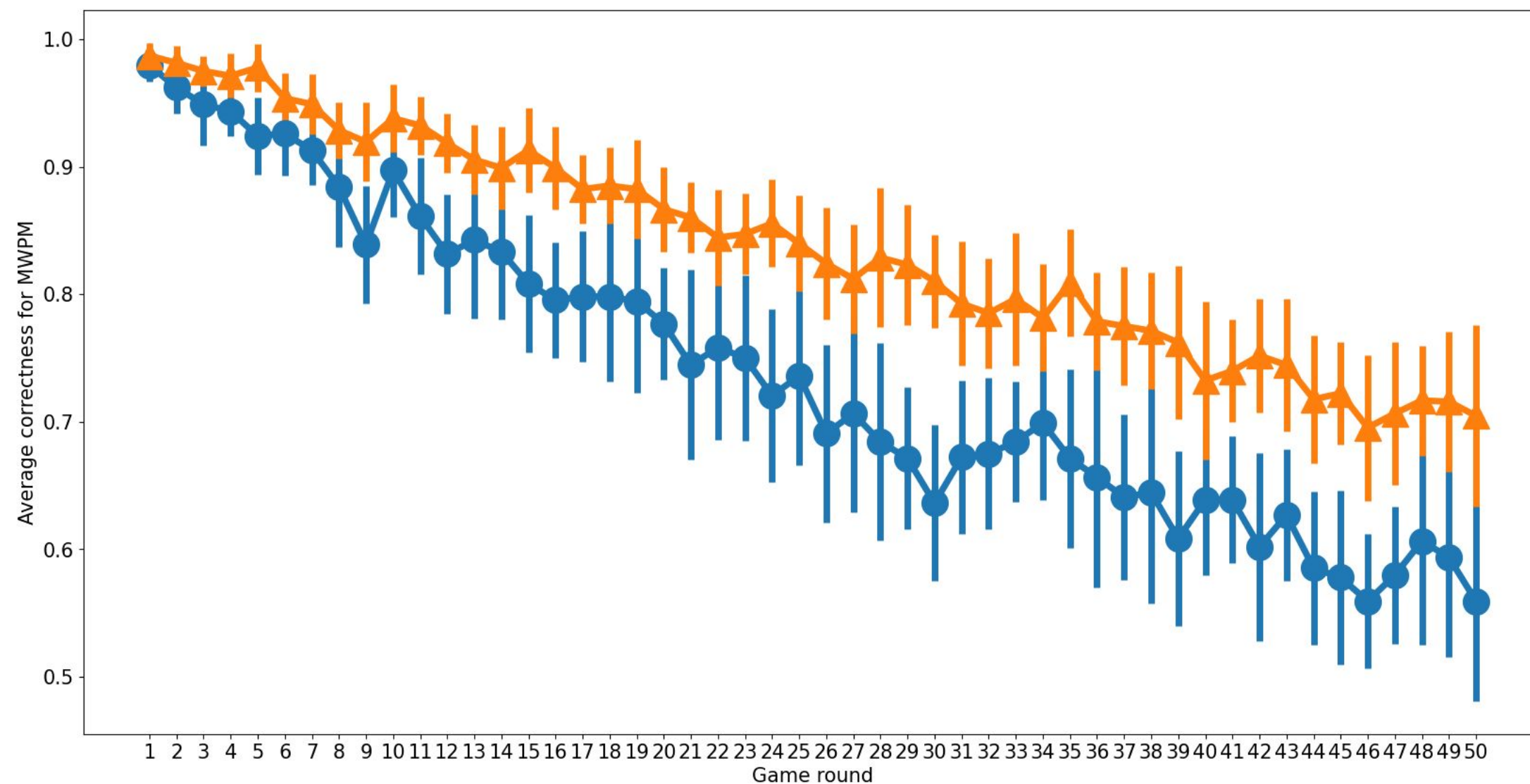
Results from `ibm_kyiv`

- 127 qubit *Eagle* device
- Peak of doom pushed out to around round 50
 - CX depth of ~100
- Matching success still at 70% for round 50 (would be ~40% for a random solution)

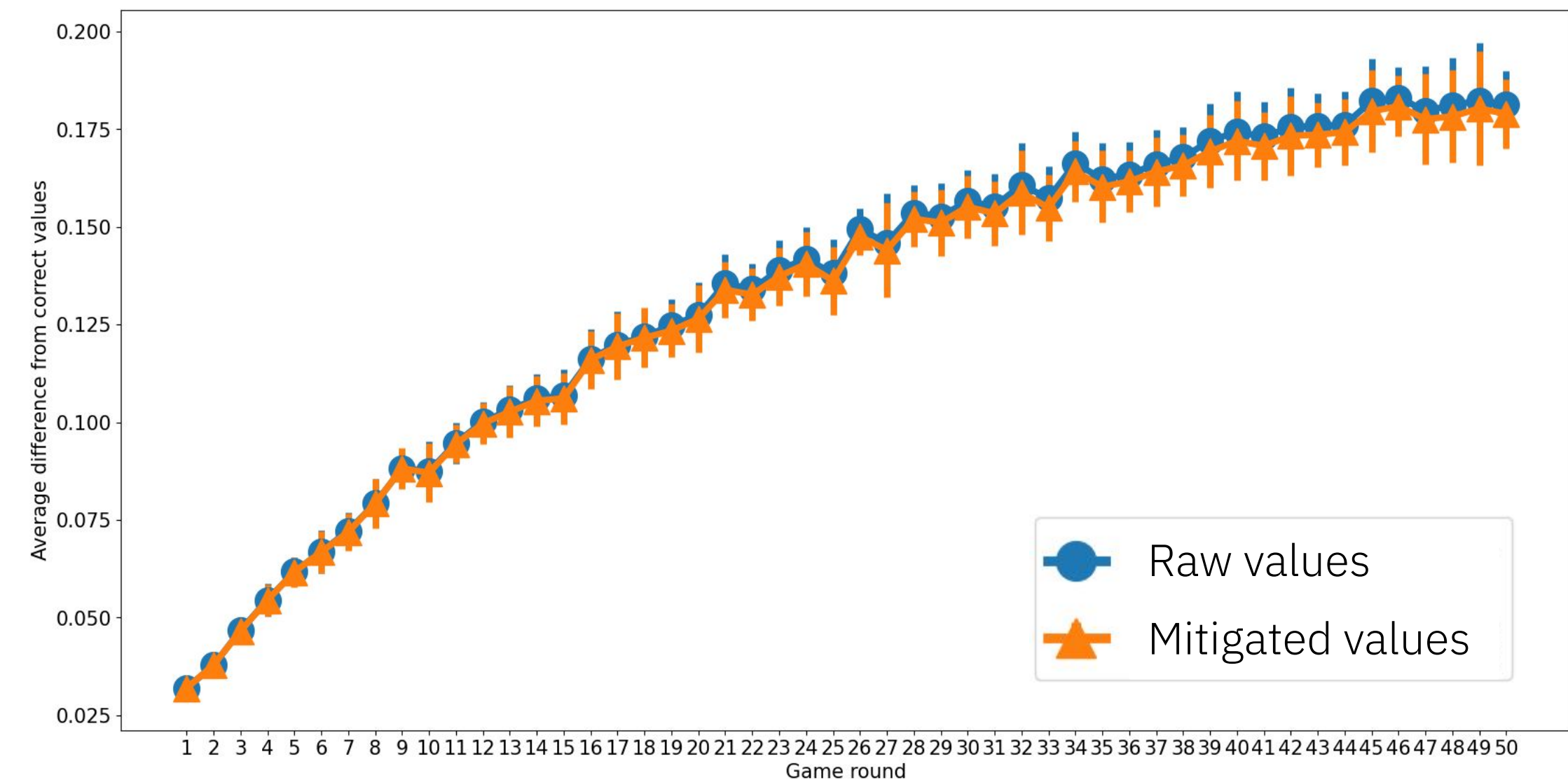
Fuzziness



Matching success



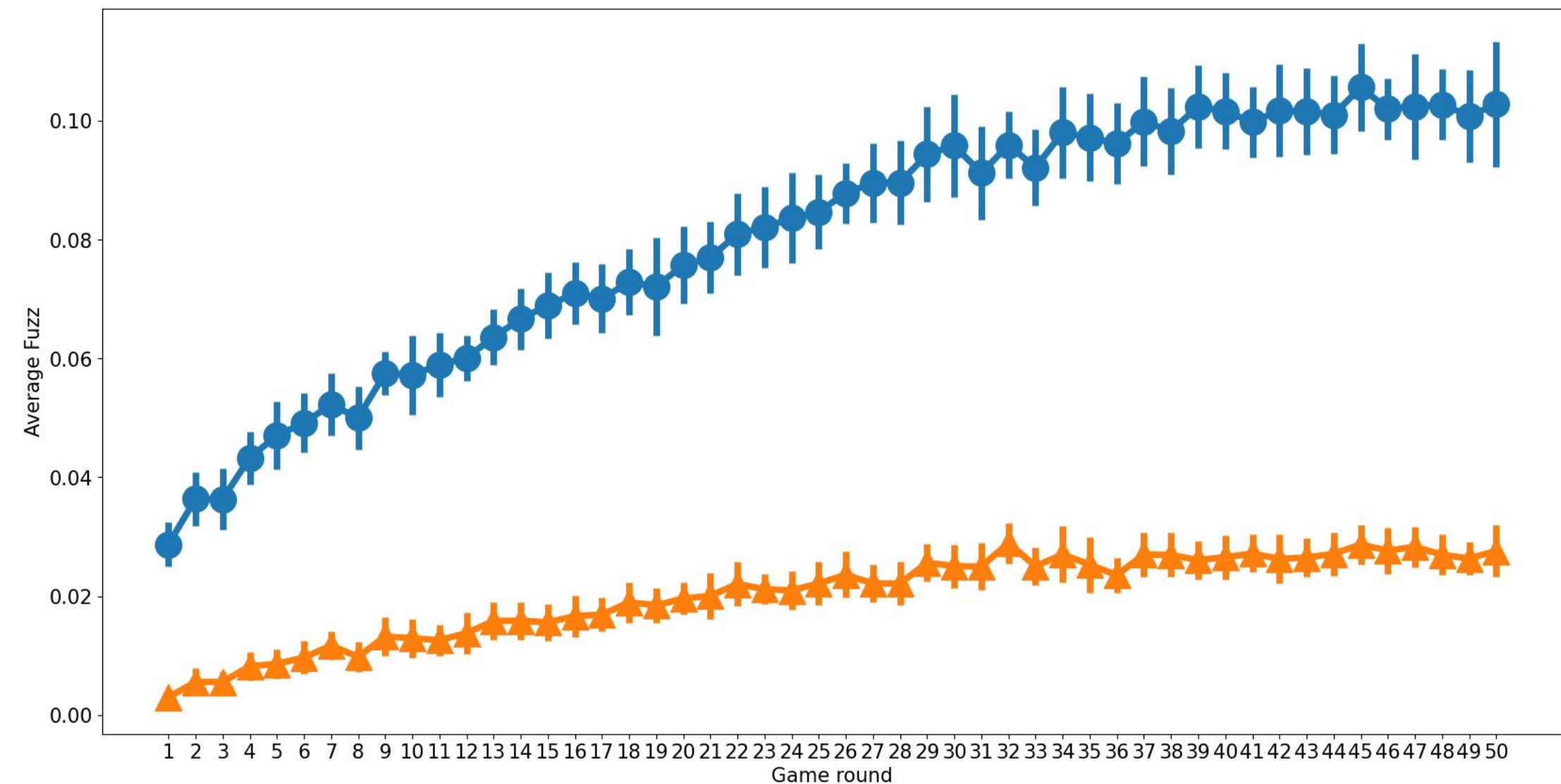
Difference



Results from `ibm_fez`

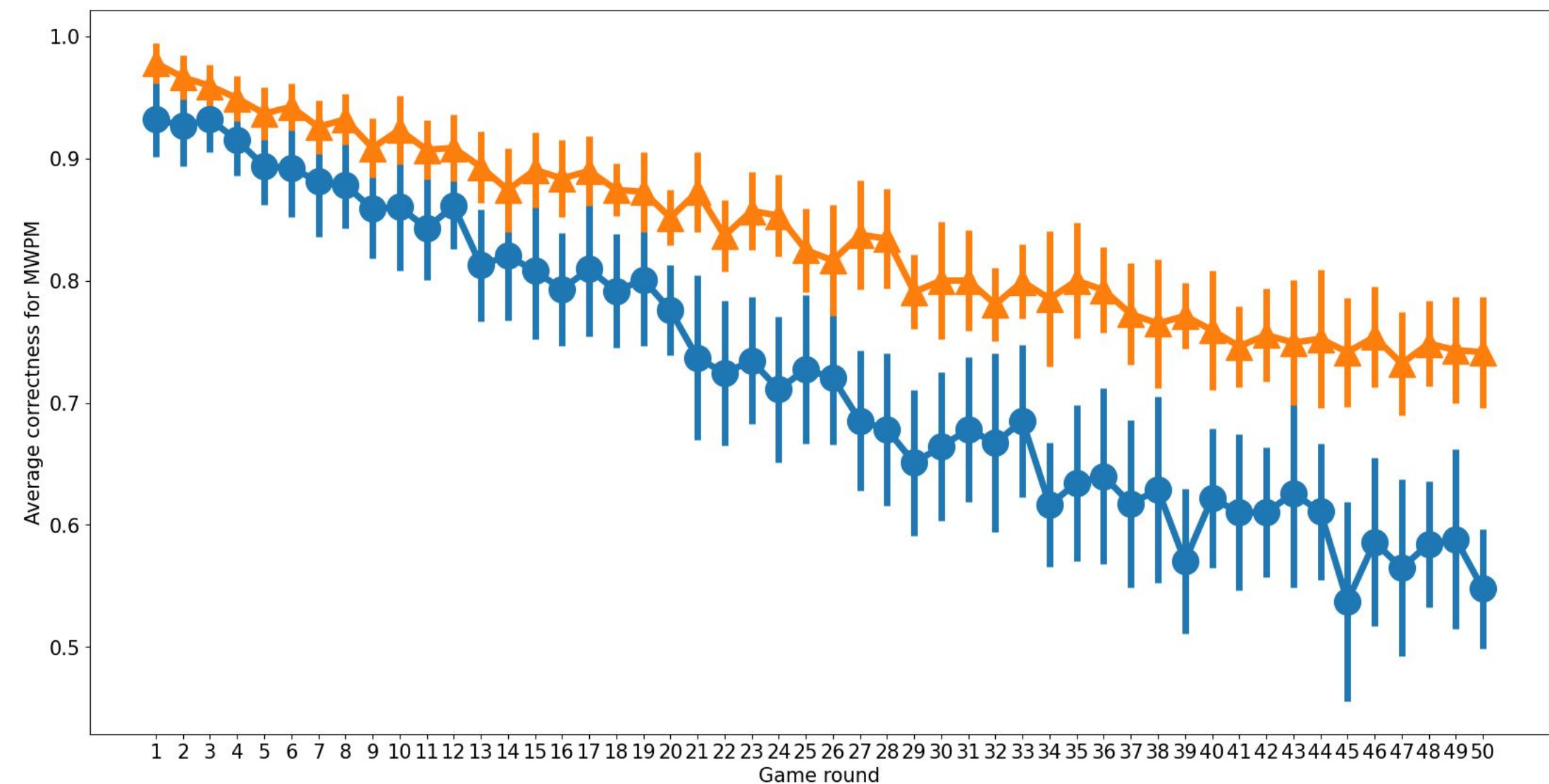
- 156 qubit *Heron* device
- Better ELPG than `ibm_torino`
- Similar behaviour to `ibm_kyiv`, but with more qubits

Fuzziness

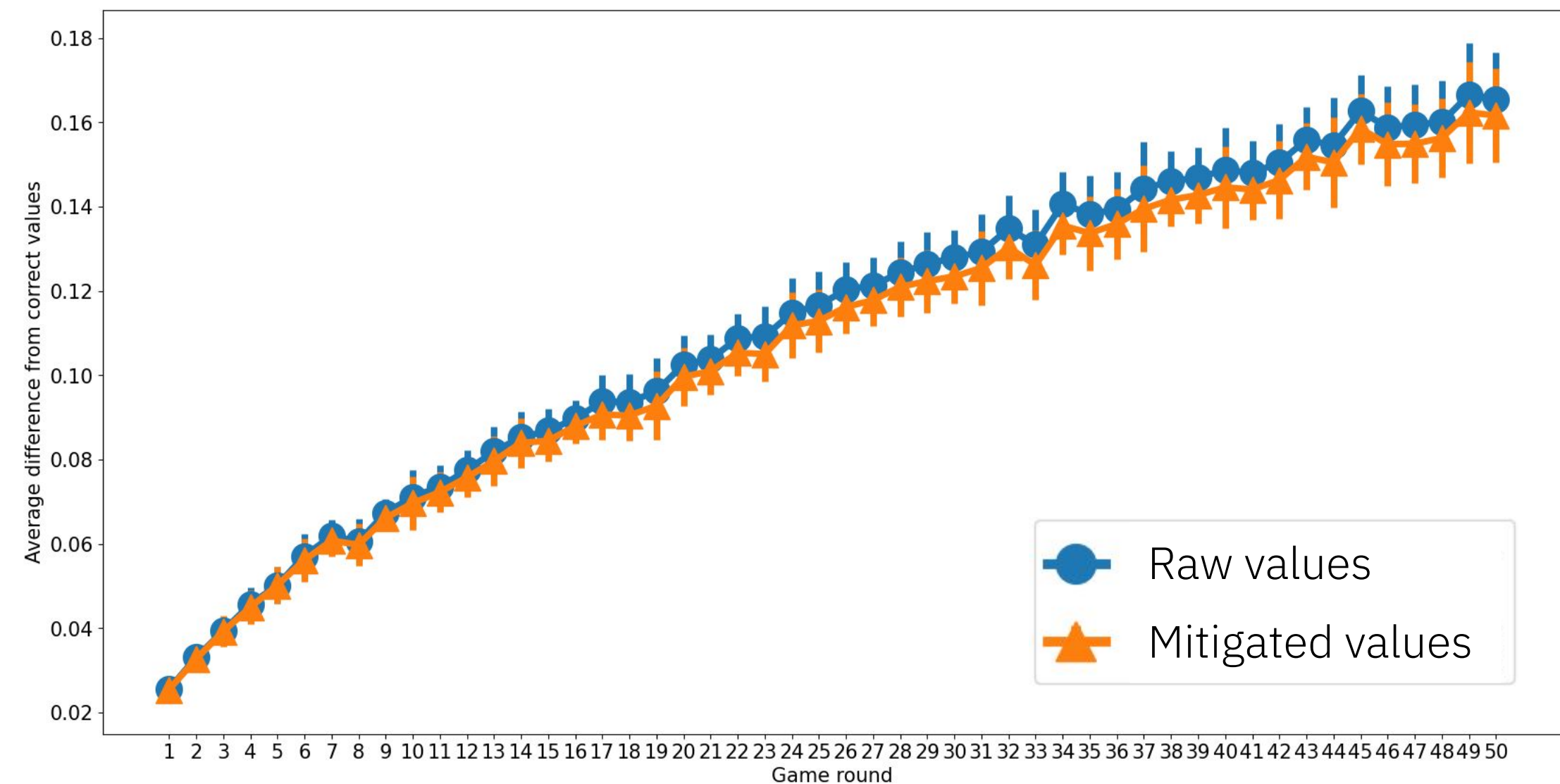


C

Matching success



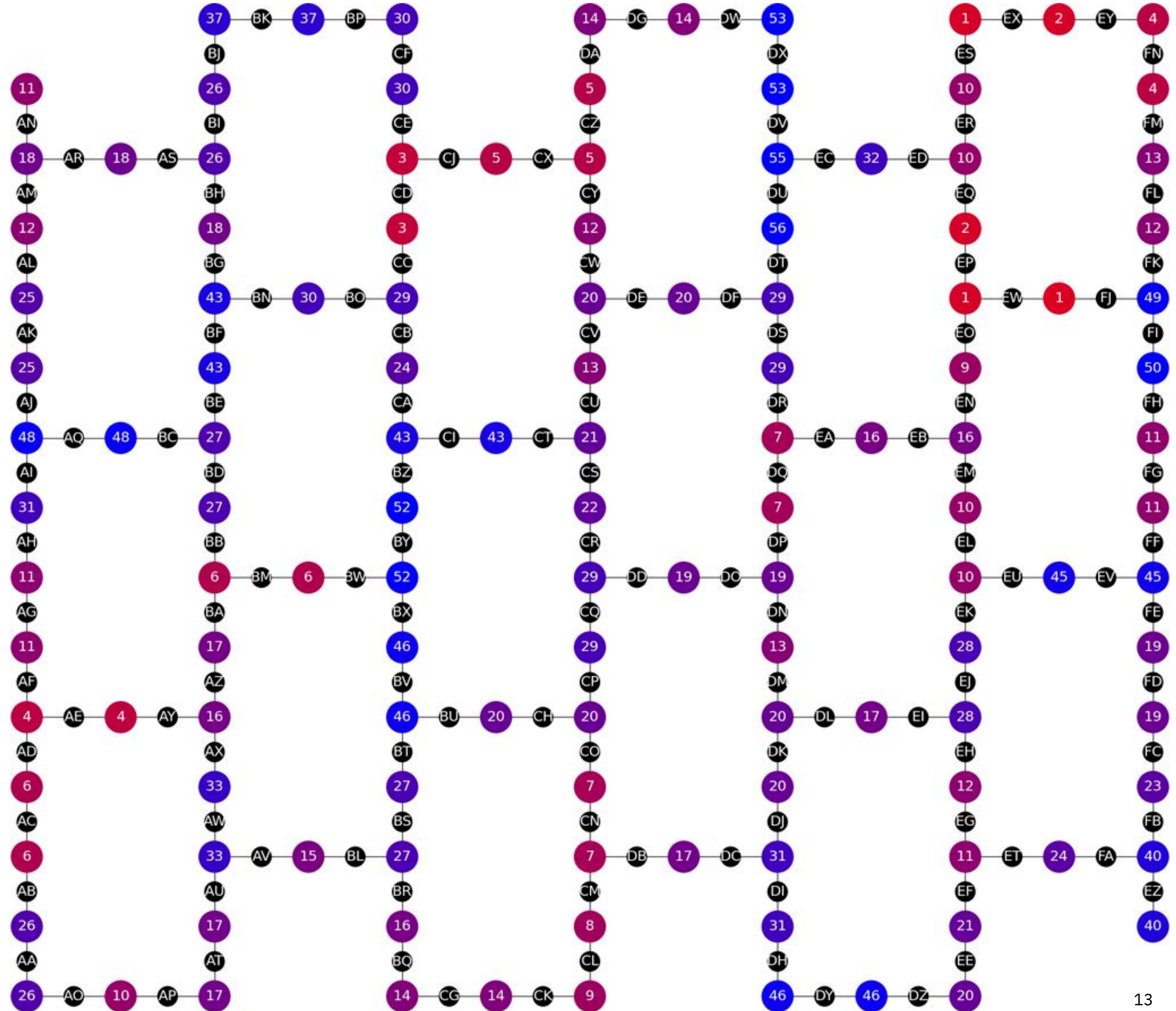
Difference



Puzzles from `ibm_kyiv`

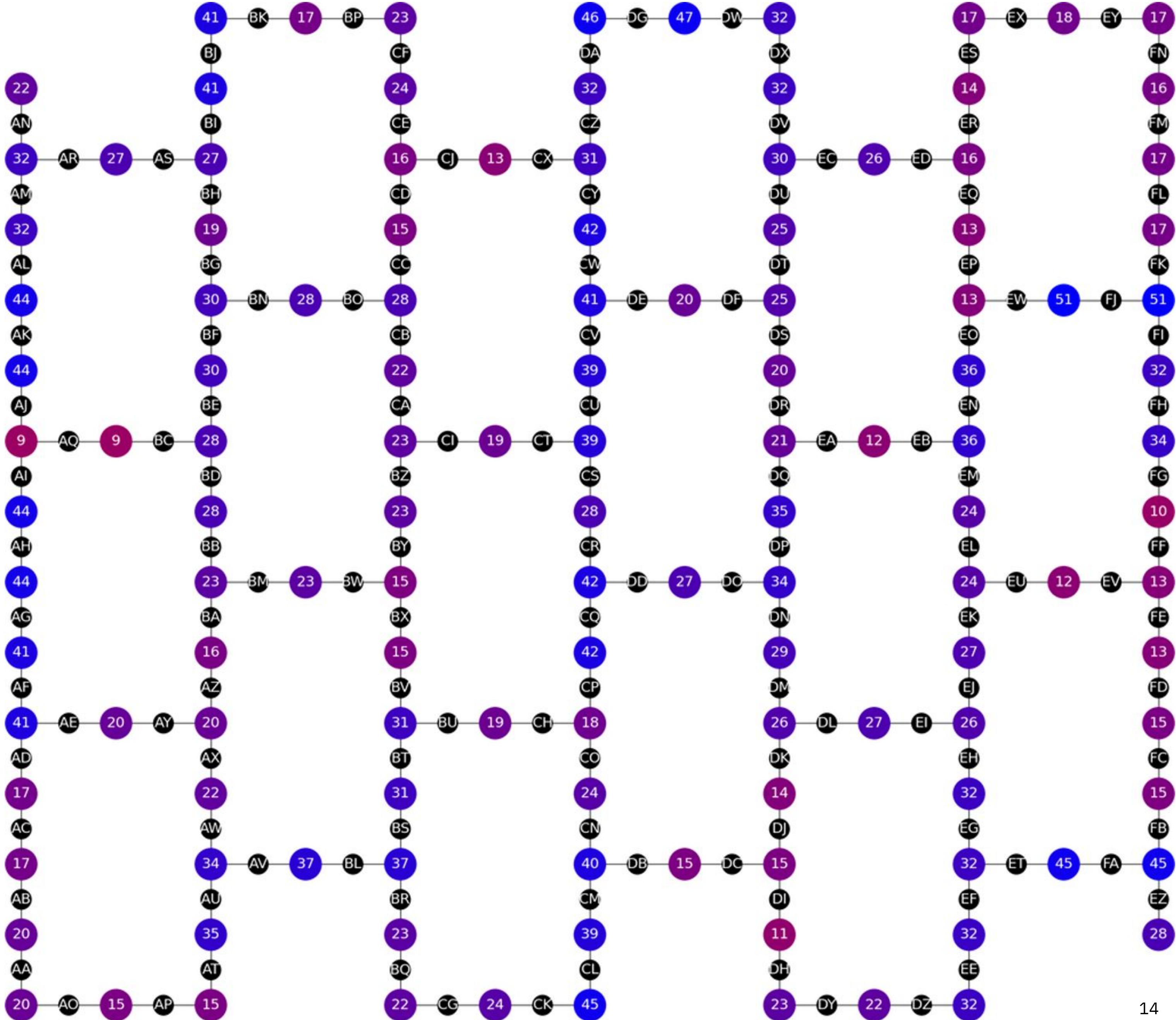
- Example of a round 1 puzzle with cleanup

Note: in this coupling graph, some qubits are left unpaired



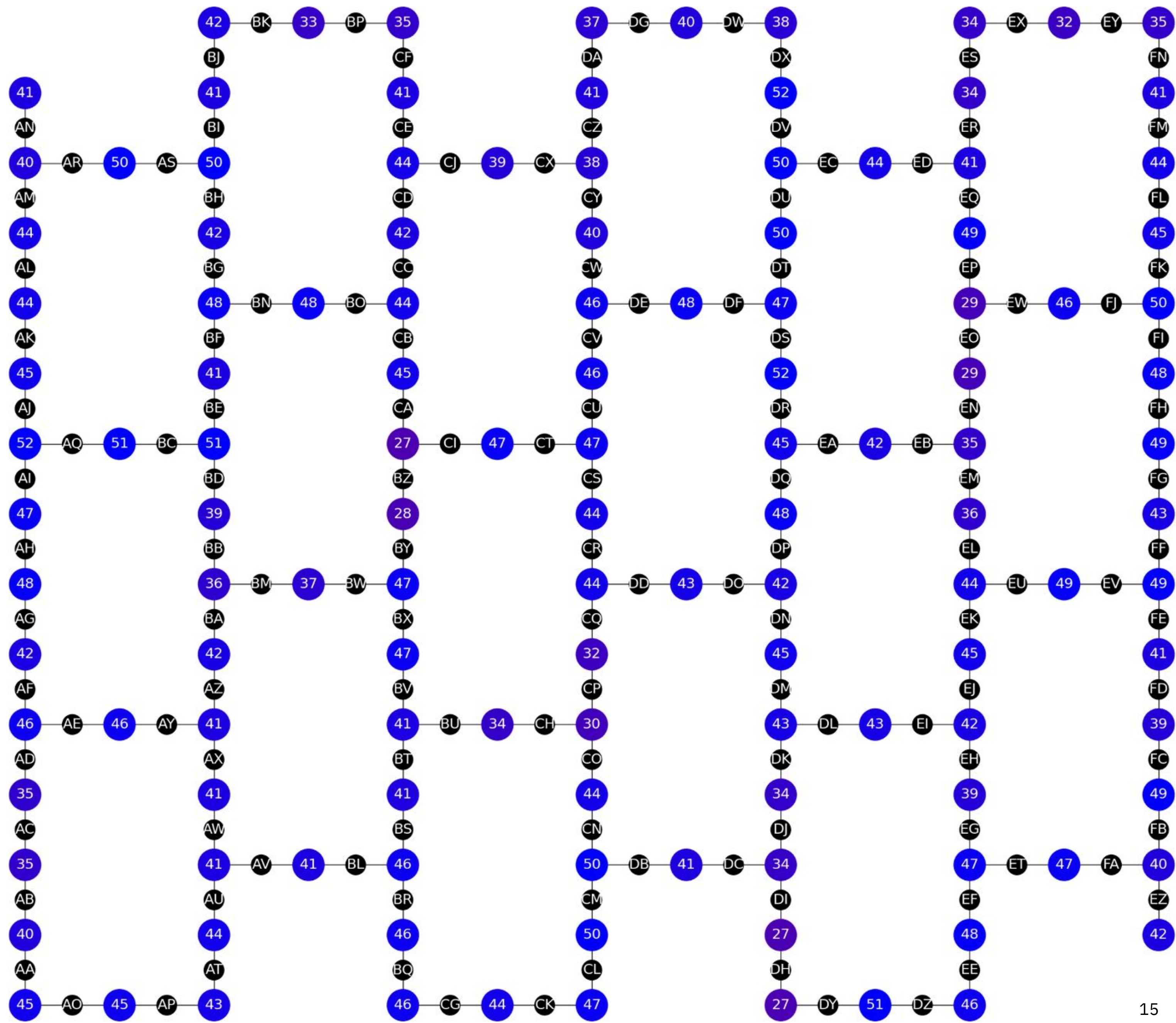
Puzzles from `ibm_kyiv`

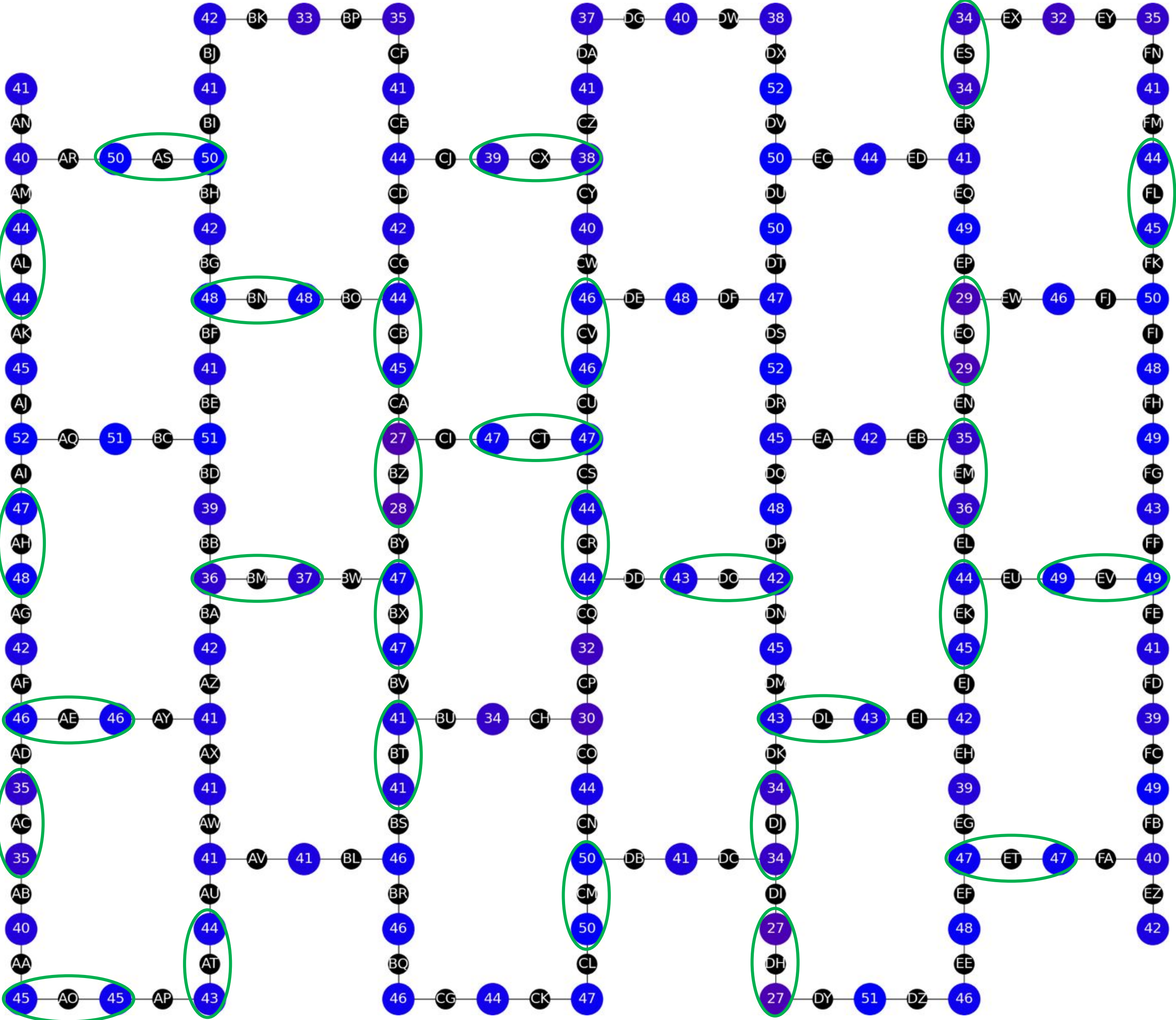
- Example of a round 10 puzzle with mitigation



Puzzles from `ibm_kyiv`

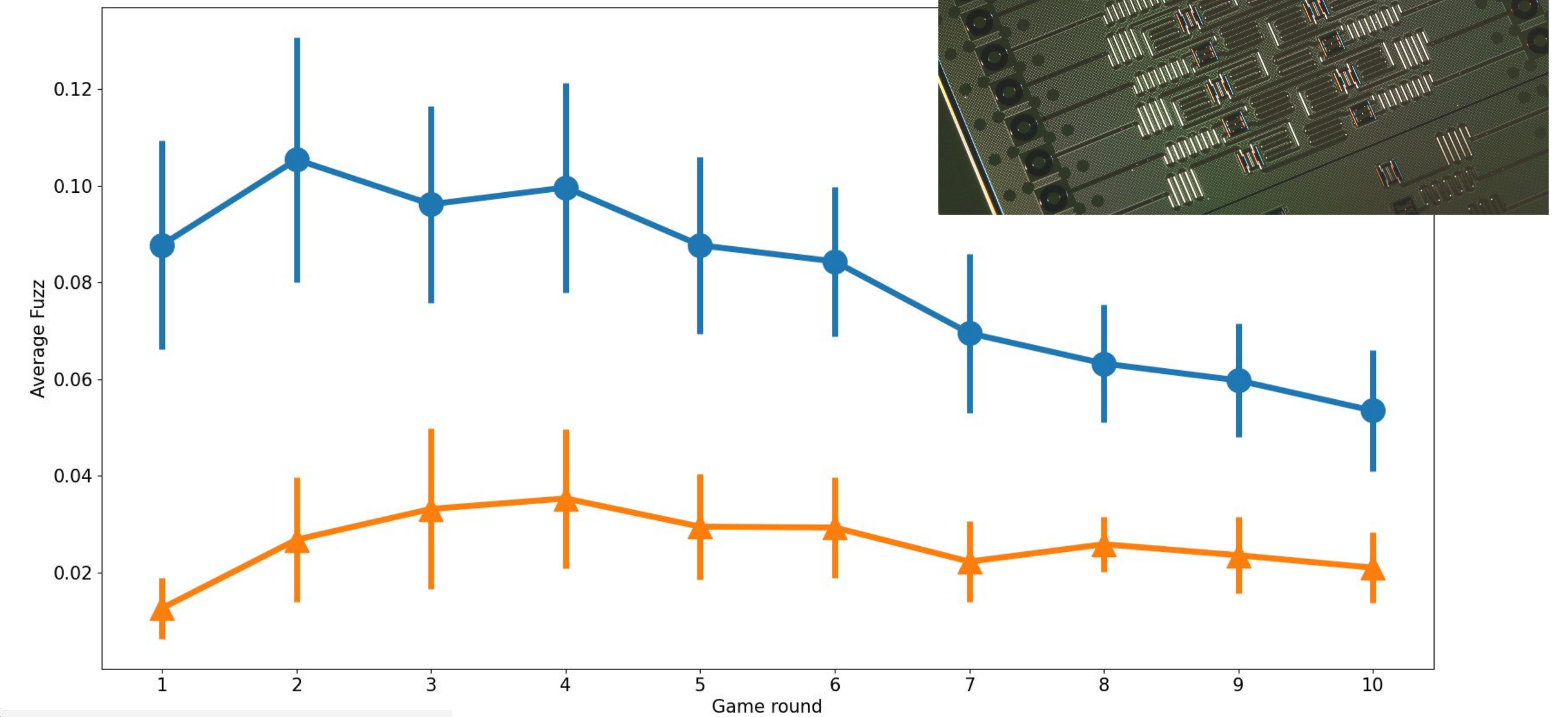
- Example of a round 50 puzzle with cleanup



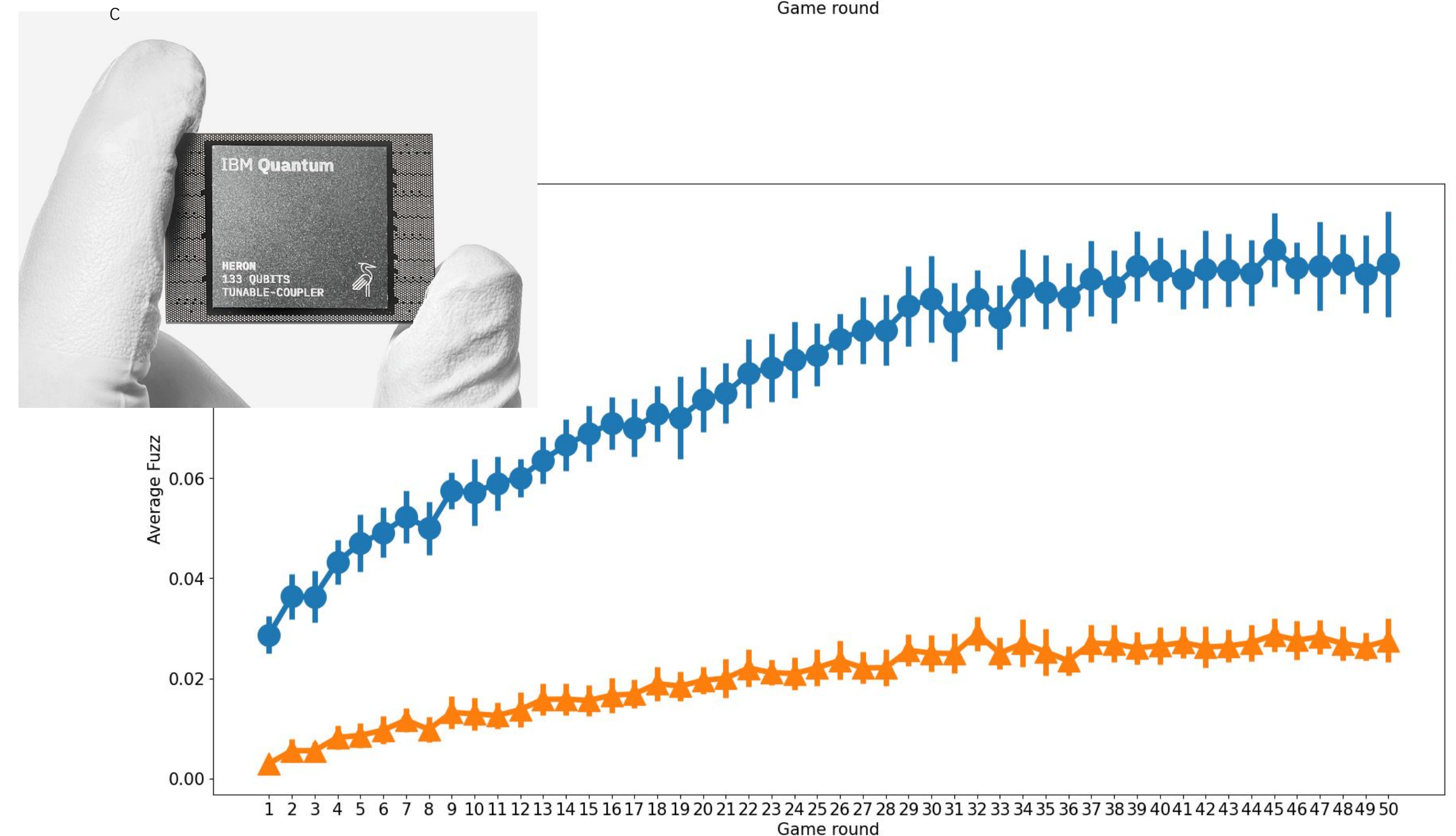


Conclusions

- We ran something that barely worked in 2018



- Now it works great with over 100 qubits!



Thanks for your
attention

